

Response to Sheikh Sameer — Clarification Note (v1.2 Review)

LocalAI TV — Integration Contract Lock · Aligned with Plan v1.3

To: Sheikh Sameer (AI Pipeline Engineer) **From:** LocalAI TV Architecture Team **Subject:** Definitive answers to your 4 open questions + confirmations 2.1/2.2 — integration contract now LOCKED **Date:** 15 May 2026 **Status:**  All questions answered. You are cleared to begin Phase 0 + Phase 1.

Dear Sameer,

Thank you for the clarification note. Your confirmations are accepted, your committed delivery plan is excellent, and your 4 open questions are exactly the right ones to close before locking the contract.

This letter answers everything definitively. **After you read this, there are zero open assumptions — you can lock the integration contract and start Phase 1.**

1. Your Confirmations — Accepted

Everything you confirmed in Section 4 of your note is accepted:

You confirmed	Status
Webhook contract (HMAC + idempotency + retries)	 Locked
Bulletin output structure per Gnana's spec	 Locked
Cancel-job endpoint (DELETE /webhook/cancel-job/:job_id)	 Locked
Callback retry policy (5 attempts, dead-letter after exhaustion)	 Locked
Direct-publish lane (admin + NotebookLM bypass)	 Locked
OpenTelemetry instrumentation (all 5 stages)	 Locked
Fallback policies (Gemini→OpenAI, TTS→Sarvam)	 Locked

Your committed delivery plan (Phase 0 this week, Phase 1 weeks 1-2, with task ownership between you and Gnana) is **approved as written**. It is now incorporated into Plan v1.3 §20.

2. Confirmations You Requested

2.1 — EC2 migration: lift-and-shift or logic changes?

CONFIRMED: Pure lift-and-shift. ZERO pipeline logic changes.

The only adjustment is a one-line performance flag, not a logic change:

```
# In your FFmpeg command(s):
- -c:v libx264 -preset fast
+ -c:v h264_nvenc -preset p4 -tune hq
```

This swaps CPU encoding for the T4 GPU's hardware encoder (NVENC). It is what takes bulletin generation from ~15 min → ~90 sec. Everything else — Gemini calls, TTS calls, Celery task structure, RabbitMQ queues, S3 read/write, build worker, maintenance worker — runs **exactly as-is**.

Migration checklist:

1. Launch EC2 g4dn.xlarge in `ap-south-1`
2. Install: CUDA 12.x + NVIDIA driver 535+ + FFmpeg with `--enable-nvenc` + Python 3.11
3. `git clone` your existing pipeline code (no code changes)
4. Swap the FFmpeg encoder flag (above)
5. Verify S3 read/write works (you already have IAM/credentials)
6. Test one bulletin: target < 2 minutes (you'll likely see ~90 sec)

2.2 — Webhook flow confirmation

CONFIRMED: Your understanding is 100% correct. No additional steps:

Admin approves

→ pg_boss worker sends POST /webhook/content-approved

(X-LocalAITV-Signature HMAC + idempotency_key in body)

→ Your FastAPI validates → returns 202 Accepted within 200ms

→ Celery processes (Gemini → TTS → FFmpeg with NVENC)

→ Upload to S3 ai-processed/<state>/<dist>/<const>/<cat>/<content_id>/

→ You POST callback → /api/webhook/ai-processed

→ Admin sets content.status = 'ready_for_bulletin'

Push-based callback. No polling required. Confirmed.

3. Your 4 Open Questions — Answered

Q1 — Webhook URL: production confirmation + staging URL?

Answer: Both provided. Environment-configurable — never hardcoded.

Environment	Webhook (Admin → You)	Callback (You → Admin)
Staging (Phase 1 testing)	<code>https://pipeline-staging.localaitv.com/webhook/content-approved</code>	<code>https://api-staging.localaitv.com/api/webhook/ai-processed</code>
Production	<code>https://pipeline.localaitv.com/webhook/content-approved</code>	<code>https://api.localaitv.com/api/webhook/ai-processed</code>

- Both routes go through Cloudflare (WAF + DDoS protection)
- `api.localaitv.com` resolves to the Admin Dashboard FastAPI on Hostinger VPS
- `pipeline.localaitv.com` resolves to your AI Pipeline FastAPI on AWS EC2
- Configure your FastAPI to read the accepted origin from an env var:

ADMIN_WEBHOOK_ALLOWED_ORIGIN=https://api-staging.localaitv.com # Phase 1

ADMIN_WEBHOOK_ALLOWED_ORIGIN=https://api.localaitv.com # Production

- **Phase 1 testing happens on staging first.** We promote to production only after end-to-end validation.

Q2 — Shared HMAC secret: how exchanged?

Answer:

Step	Detail
Generation	Architecture team generates once: <code>openssl rand -hex 32</code> (64-char hex)
Admin side storage	Supabase Vault (encrypted at rest)
Your side storage	AWS Secrets Manager (you're on AWS EC2 now — native fit) OR a systemd environment file with <code>chmod 600</code>
Exchange channel	1Password / Bitwarden shared vault — NOT email, Slack, or git
Your env var	<code>WEBHOOK_HMAC_SECRET</code> (read at FastAPI startup)
Rotation	New secret generated → both sides accept BOTH old + new for a 24h grace window → old retired. We provide a rotation runbook. You will get 48h advance notice before any rotation.

Verification logic on your side (use the **raw request body**, not re-serialized JSON):

```
raw_body = await request.body()
expected = hmac.new(WEBHOOK_HMAC_SECRET.encode(), raw_body, hashlib.sha256).hexdigest()
if not hmac.compare_digest(expected, request.headers["X-LocalAITV-Signature"]):
    return JSONResponse({"error": "invalid_signature"}, status_code=401)
```

Q3 — Callback bearer token: who generates, how rotated?

Answer:

Aspect	Detail
Who generates	Admin Dashboard generates it — it's the Admin's API you call, so Admin owns the credential
Token	<code>openssl rand -hex 32</code>
Admin side storage	Supabase Vault
Your side storage	AWS Secrets Manager as <code>ADMIN_CALLBACK_TOKEN</code>
Exchange channel	Same 1Password vault as the HMAC secret
You send it as	Authorization: Bearer <ADMIN_CALLBACK_TOKEN> on the <code>/api/webhook/ai-processed</code> POST
Rotation	Admin generates new token → Admin's callback endpoint accepts BOTH old + new for 24h → old revoked. Admin notifies your team 48h before rotation.

Never hardcode. Read from env at startup.

Q4 — If callback returns 4xx, what does Admin do with content status?

Answer (this is the most important contract clarification — standard HTTP semantics):

Callback response	Your action	Admin Dashboard action
2xx success	Mark job complete in <code>ai_processing_jobs</code>	<code>content.status = 'ready_for_bulletin'</code>
5xx (Admin temporarily down)	RETRY — 5 attempts, exp backoff (1s→5s→30s→5min→30min). Transient.	Recovers on retry; pg_cron watchdog also catches it
4xx (bad payload from your side)	DO NOT RETRY — permanent. Write to <code>ai_callbacks_outbox</code> (dead-letter). Log + Sentry alert.	Content stays in <code>ai_processing</code>
Timeout / no response	Treat like 5xx — retry	pg_cron watchdog catches after 30 min

Your specific question — does content stay in `ai_processing` forever?

No. The Admin Dashboard's `pg_cron` watchdog runs **every 5 minutes**:

1. Finds content in `ai_processing` state for **> 30 minutes** with no successful callback
2. Marks `content.status = 'callback_lost'`
3. Alerts super admin
4. Super admin manually reconciles:
 - **Video IS in S3** (`ai-processed/...`)? → super admin clicks "Confirm processed" → `ready_for_bulletin` (your processing succeeded; only the callback wire failed)
 - **Video NOT in S3?** → super admin clicks "Re-queue" → content reverts to `approved` → new webhook fires with a fresh `idempotency_key` (so you won't reject it as duplicate)

Implication for your error handling: On a 4xx from the callback, log + dead-letter, but **do NOT mark the content as failed on your side**. Your video may be perfectly fine in S3 — the Admin watchdog will reconcile. Just preserve the S3 output and the `ai_callbacks_outbox` row so reconciliation can succeed.

This logic is documented in Plan v1.3 §15.7.

4. Your Committed Delivery Plan — Approved & Incorporated

Your Phase 0 + Phase 1 task tables (with you/Gnana ownership) are accepted as-is and now appear in Plan v1.3 §20. Summary:

Phase 0 (this week) — owners: Sameer + Gnana

- Launch EC2 g4dn.xlarge, install CUDA + FFmpeg-NVENC + Python 3.11 (Sameer)
- Migrate pipeline code from Hostinger VPS to EC2 (Gnana)
- Test bulletin generation < 2 min (Sameer + Gnana)
- Verify S3 read/write from EC2 (Sameer)

Phase 1 (Week 1-2) — webhook adapter (you + Gnana)

- All 11 tasks in your table are approved as written
 - "Existing pipeline requires zero changes — purely the integration adapter layer" — **confirmed correct**
-

5. You Are Cleared to Begin

✅ All confirmations accepted ✅ All 4 questions answered definitively ✅ Your delivery plan approved ✅ Integration contract is now **LOCKED**

Phase 0 (EC2 migration) can start immediately — it does not depend on the Admin Dashboard build. The webhook adapter (Phase 1) can begin once Phase 0 verifies the EC2 instance works.

The HMAC secret and callback token will be delivered to you via 1Password before Phase 1 begins (Admin team will set up the shared vault and invite you + Gnana).

6. Closing

Your engineering discipline — confirming understanding before building, asking the precise questions, committing to a clear ownership plan — is exactly how this should be done. The contract is unambiguous now.

If anything in these answers needs adjustment, reply with specifics. Otherwise, proceed with Phase 0 this week.

With respect,

LocalAI TV Architecture Team LocalAI Media Network Pvt Ltd CIN: U63910KA2025PTC212593 Hyderabad, India

Plan v1.3 (separate document) reflects every answer above in §15, §16, and §20.